

```
# =====
# MINI PROJECT
# Customer Segmentation + Purchase Prediction
# Supervised + Unsupervised Learning
# =====

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# =====
# STEP 1: CREATE DATASET
# =====

np.random.seed(42)

n = 300

age = np.random.randint(18, 60, n)
income = np.random.randint(20000, 120000, n)
spending_score = np.random.randint(1, 100, n)

purchase = []

for i in range(n):
    score = 0

    if income[i] > 60000:
        score += 1

    if spending_score[i] > 50:
        score += 1

    if age[i] < 40:
        score += 1

    purchase.append(1 if score >= 2 else 0)

df = pd.DataFrame({
    'Age': age,
    'Income': income,
    'SpendingScore': spending_score,
    'Purchase': purchase
})

print("\nDataset Sample:")
print(df.head())

# =====
# STEP 2: DATA PREPROCESSING
# =====

features = df[['Age', 'Income', 'SpendingScore']]

scaler = StandardScaler()
scaled_features = scaler.fit_transform(features)

# =====
# STEP 3: UNSUPERVISED LEARNING
```

```

# K-MEANS CLUSTERING
# =====

kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)

clusters = kmeans.fit_predict(scaled_features)

df['Cluster'] = clusters

print("\nCluster Counts:")
print(df['Cluster'].value_counts())

# =====
# STEP 4: VISUALIZATION
# =====

plt.figure(figsize=(8,6))

plt.scatter(
    df['Income'],
    df['SpendingScore'],
    c=df['Cluster'],
    cmap='viridis'
)

plt.xlabel("Income")
plt.ylabel("Spending Score")
plt.title("Customer Segmentation using K-Means")
plt.colorbar(label="Cluster")
plt.show()

# =====
# STEP 5: SUPERVISED LEARNING
# PURCHASE PREDICTION
# =====

X = df[['Age', 'Income', 'SpendingScore']]
y = df['Purchase']

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

model = LogisticRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("\nModel Accuracy:")
print(f"{accuracy * 100:.2f}%")

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# =====
# STEP 6: NEW CUSTOMER PREDICTION
# =====

```

```
new_customer = pd.DataFrame({
    'Age': [28],
    'Income': [85000],
    'SpendingScore': [75]
})

prediction = model.predict(new_customer)

print("\nNew Customer Data:")
print(new_customer)

if prediction[0] == 1:
    print("\nPrediction: Customer WILL Purchase")
else:
    print("\nPrediction: Customer WILL NOT Purchase")

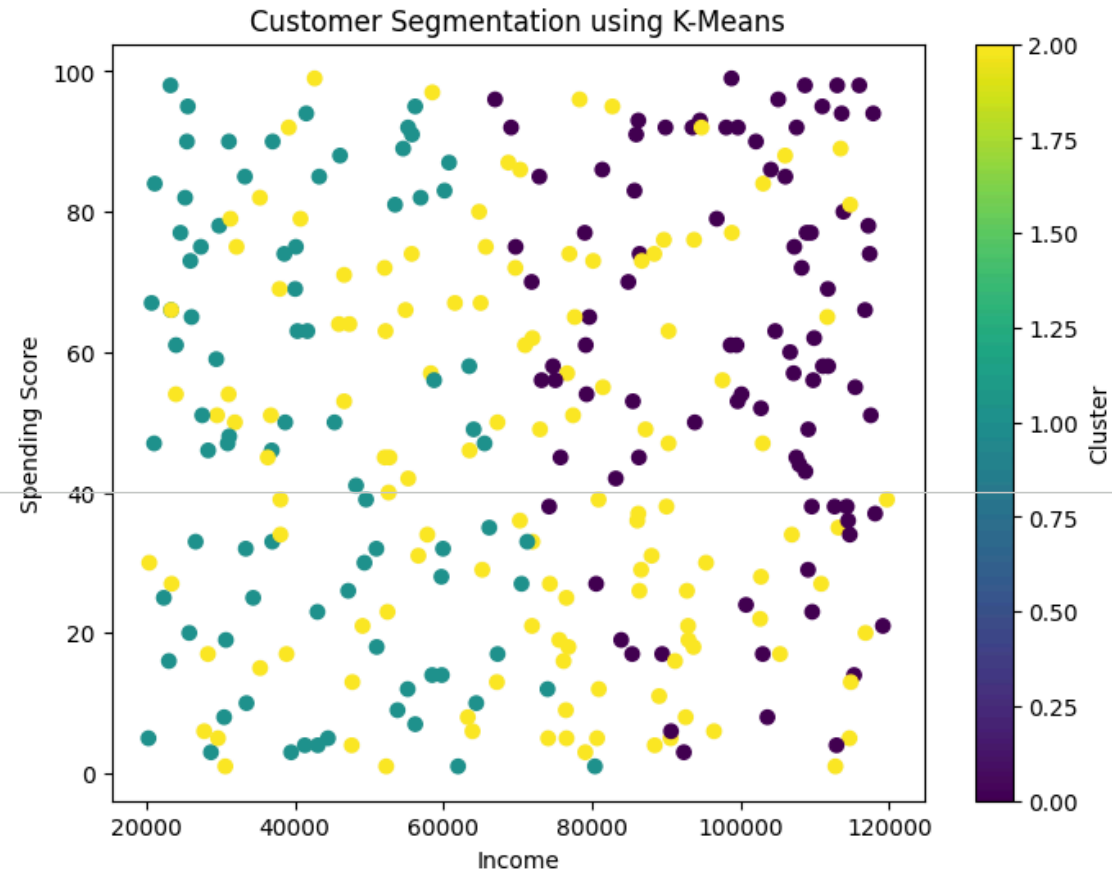
# =====
# STEP 7: SAVE DATASET
# =====

df.to_csv("customer_dataset.csv", index=False)

print("\nDataset saved as customer_dataset.csv")
print("Project Completed Successfully!")
```

```
Dataset Sample:
  Age  Income  SpendingScore  Purchase
0   56   59790             14         0
1   46   25600             95         0
2   32  114660              5         1
3   25  113179             35         1
4   38   60764             87         1
```

```
Cluster Counts:
Cluster
2      128
0       89
1       83
Name: count, dtype: int64
```



Model Accuracy:
78.33%

```
Classification Report:
              precision    recall  f1-score   support

0               0.92        0.68       0.78         34
1               0.69        0.92       0.79         26
```